WRITE DATASET AND OUTPUTS ON LEFT SIDE OF YOUR RECORD

Experiment: 4

Aim: Demonstrate decision tree algorithm for a classification problem and perform parameter tuning for better results.

A Decision Tree Classifier is a supervised machine learning algorithm used to classify data into different categories by splitting the dataset based on feature values using measures like Gini Index or Entropy.

- Used for **classification problems**.
- Data is split into branches based on feature conditions.
- Tree structure contains:
  - Root node
  - Decision nodes
  - Leaf nodes (class labels)
- Uses **Gini Index or Entropy** to select best split.
- Easy to understand and visualize.
- May suffer from **overfitting** if tree grows too deep.
- Parameter tuning improves model performance.

Entropy

$$Entropy(S) = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

Where:

- p_i= probability of class i

Information Gain

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Where:

- S= dataset
- A= attribute
- S_v= subset of S for attribute value v

Gini Index

$$Gini = 1 - \sum_{i=1}^{n} p_i^2$$

Algorithm

Step 1: Load the dataset.
Step 2: Separate features (X) and target variable (Y).
Step 3: Split the dataset into training and testing sets.
Step 4: Select the best attribute using Entropy or Gini Index.
Step 5: Split the dataset based on selected attribute.
Step 6: Repeat splitting until stopping condition is met.
Step 7: Predict class labels for test data and evaluate accuracy.

Program:

Decision Tree for Classification

Program

```
#import Libraries

import numpy as np

import matplotlib.pyplot as plt


from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


#loading and splitting dataset

iris = load_iris()

X = iris.data        # Features

y = iris.target      # Target labels

print("Feature Names:", iris.feature_names)

print("Target Names:", iris.target_names)


# splitting data
```

```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=10)

#model training
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy (Before Tuning):", accuracy_score(y_test, y_pred))

# Finding Best Parameters and parameter tuning
param_grid = {
    'max_depth': [2, 3, 4, 5],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'criterion': ['gini', 'entropy']
}

grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv=5 )

grid.fit(X_train, y_train)
print("Best Parameters:", grid.best_params_)

#Tuned Model
best_model = grid.best_estimator_
y_pred_tuned = best_model.predict(X_test)
print("Accuracy (After Tuning):", accuracy_score(y_test, y_pred_tuned))
print(confusion_matrix(y_test, y_pred_tuned))
```

```
# plotting Tree
plt.figure(figsize=(12,8))
plot_tree(
    best_model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
)
plt.show()
```

Experiment: 5

Aim: Demonstrate decision tree algorithm for a regression problem.

Decision Tree Algorithm for Regression

Decision Tree Regressor is a supervised learning algorithm used to predict continuous numerical values by dividing data into smaller subsets and predicting the average value in each region.

- Used for regression problems.
- Predicts continuous values.
- Splits dataset based on minimizing variance or Mean Squared Error.
- Each leaf node contains predicted value.

Mean Squared Error (MSE)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Where:

- y_i= actual value
- $\hat{y}_i$= predicted value

Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

Coefficient of Determination (R^2)

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Where:

- SS_{res}= residual sum of squares
- SS_{tot}= total sum of squares

Algorithm

Step 1: Load the dataset.
Step 2: Separate features (X) and target variable (Y).
Step 3: Split dataset into training and testing sets.
Step 4: Select best split by minimizing Mean Squared Error.
Step 5: Divide data into subsets based on selected split.
Step 6: Repeat splitting until stopping criteria is met.
Step 7: Predict continuous values and evaluate using MSE, RMSE, or R^2.

Program:

```
# Import required libraries

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt


from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeRegressor

from sklearn.metrics import mean_squared_error, r2_score


# Load Iris dataset

iris = load_iris()


# Features (X) and Target (y)

X = iris.data[:, [0,1,3]]   # Sepal length, sepal width, petal width

y = iris.data[:, 2]         # Petal length (continuous value)



# Split dataset into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Decision Tree Regressor model
model = DecisionTreeRegressor(random_state=42)

# Train the model
model.fit(X_train, y_train)

# Predict continuous values
y_pred = model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("R² Score:", r2)
```

Experiment: 6

Aim: Apply Random Forest algorithm for classification and regression.

Random Forest is an ensemble learning algorithm that constructs multiple decision trees using random subsets of data and features, and combines their outputs to improve prediction accuracy and reduce overfitting.Random Forest is an ensemble method based on bagging (Bootstrap Aggregation).

- It builds multiple decision trees instead of one.
- Each tree is trained on a random subset of data.
- Random feature selection is applied at each split.

- Final prediction is obtained by:
    - Majority voting for classification
    - Average prediction for regression
- Helps in reducing variance and overfitting.

Important Formulas

Classification Prediction

$$\hat{y} = Mode(T_1(x), T_2(x), \ldots, T_n(x))$$

Where:
$T_i(x)$= prediction of the i^{th}tree

Regression Prediction

$$\hat{y} = \frac{1}{n}\sum_{i=1}^{n} T_i\,(x)$$

Where:
n= number of decision trees.

Algorithm

Step 1: Load the dataset.
Step 2: Split the dataset into training and testing sets.
Step 3: Generate bootstrap samples from the training data.
Step 4: Train multiple decision trees using random subsets of features.
Step 5: Repeat the process to build many trees.
Step 6: Combine predictions from all trees.

- For classification - majority voting.
- For regression - average prediction.
  Step 7: Evaluate model performance using appropriate metrics.

6A. Random Forest Classification

Program:

```
# Import Required Libraries

import numpy as np

import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load the Dataset
iris = load_iris()

# Convert to DataFrame (easier to understand)
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target)
print("First 5 Rows of Dataset:\n")
print(X.head())


# Split the Dataset (80% Training, 20% Testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42)
print("\nTraining Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])


# Create Random Forest Model
rf_model = RandomForestClassifier(
    n_estimators=100,      # number of trees
    max_depth=None,        # grow trees fully
    max_features='sqrt',   # random feature selection
    bootstrap=True,        # use bootstrapping
    random_state=42
)
```

```
# Train the Model
rf_model.fit(X_train, y_train)

# Make Predictions
y_pred = rf_model.predict(X_test)

# Evaluate Model Performance
print("\nModel Performance:")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=iris.target_names))


# Predict New Sample
new_flower = [[5.1, 3.5, 1.4, 0.2]]
prediction = rf_model.predict(new_flower)
print("\nNew Sample Prediction:",
    iris.target_names[prediction[0]])
```

6B. Random Forest Regression

Program:

```
# Import Required Libraries
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

```
# Load the Dataset
housing = fetch_california_housing()

# Convert to DataFrame (for better understanding)
X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = pd.Series(housing.target)

print("First 5 Rows of Dataset:\n")
print(X.head())

# Split the Dataset (80% Training, 20% Testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
print("\nTraining Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])


# Create Random Forest Regressor Model
rf_model = RandomForestRegressor(
    n_estimators=100,   # number of trees
    max_depth=None,     # grow trees fully
    random_state=42
)

# Train the Model
rf_model.fit(X_train, y_train)
```

```
# Make Predictions
y_pred = rf_model.predict(X_test)

# Evaluate Model Performance
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("\nModel Performance:")
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("R2 Score:", r2)


# Predict One Sample from Test Set
sample_index = 0
sample = X_test.iloc[[sample_index]]
prediction = rf_model.predict(sample)
print("\nActual Value:", y_test.iloc[sample_index])
print("Predicted Value:", prediction[0])
```